

Kitap Satış Projesi

Hazırlayan: Zühre Özen

Öğrenci No: 23253075

Ders: Yazılım Mühendisliği

Proje Adı: Kitap Satış Projesi

GitHub Proje Linki: <https://github.com/kullaniciAdi/proje-adi>

1. Proje Özeti

Bu proje, **frontend/** ve **backend/** olmak üzere iki ayrı katmandan oluşan bir kitap satış uygulamasıdır. Uygulama; müşteri ve yönetici (admin) rollerini destekler, kitap listesi, sepet yönetimi ve satın alma akışını içerir.

Projenin temel amacı, Yazılım Mühendisliği dersi kapsamında gerçek bir e-ticaret altyapısını demo olarak sunmaktır. Backend NestJS ile API olarak tasarlandı. Frontend ise React + TypeScript + Vite kullanılarak geliştirildi.

2. Ödevi Çalıştırmak İçin Gerekenler

2.1. Gereksinimler

- Node.js 18 veya üzeri
- npm
- PostgreSQL
- İki ayrı terminal penceresi

2.2. PostgreSQL Hazırlığı

Backend uygulaması PostgreSQL bağlantısı üzerinden çalışır. Varsayılan ayarlar **backend/src/app.module.ts** içinde tanımlanmıştır:

- Host: **localhost**
- Port: **5432**
- Kullanıcı: **postgres**
- Parola: **12345**
- Veritabanı: **Kitap_Satisi**

Eğer farklı bir veritabanı kullanıyorsanız, bu ayarları **backend/src/app.module.ts** dosyasında güncelleyin.

2.3. Veritabanı Oluşturma

PostgreSQL sunucusu çalışırken aşağıdaki SQL komutunu çalıştırın:

```
CREATE DATABASE "Kitap_Satisi";
```

2.4. Backend Çalıştırma Adımları

1. Terminalde proje kök dizinine gidin:

```
cd backend
```

2. Bağımlılıkları yükleyin:

```
npm install
```

3. Backend sunucusunu başlatın:

```
npm run start:dev
```

Backend sunucusu başarıyla açıldığında genellikle <http://localhost:3000> adresinde çalışır.

2.5. Frontend Çalıştırma Adımları

1. Yeni bir terminal penceresi açın ve **frontend** dizinine geçin:

```
cd frontend
```

2. Bağımlılıkları yükleyin:

```
npm install
```

3. Frontend uygulamasını başlatın:

```
npm run dev
```

Frontend uygulaması tipik olarak <http://localhost:5173> adresinde açılır.

2.6. Çalıştırma Önceliği

1. PostgreSQL servisinin çalıştığından emin olun.
2. **backend** klasöründe `npm install` çalıştırın.
3. **backend** tarafını `npm run start:dev` ile başlatın.
4. **frontend** klasöründe `npm install` çalıştırın.
5. **frontend** tarafını `npm run dev` ile başlatın.
6. Tarayıcıyı açıp <http://localhost:5173> adresine gidin.

Bu bölüm, ödevi çalıştırmak için en kritik adımdır. İlk önce veritabanı ve backend çalışmalı, ardından frontend açılmalıdır.

3. Projenin Detaylı Anlatımı

3.1. Genel Mimarisi

Proje iki ana kısımdan oluşur:

- **backend/**: NestJS tabanlı REST API
- **frontend/**: React + TypeScript tabanlı kullanıcı arayüzü

Bu yapı sayesinde iş mantığı sunucu tarafında çalışırken, kullanıcı etkileşimi modern bir SPA ile sağlanır.

3.2. Backend Detayları

Backend **backend/src/** altında modüler yapıda organize edilmiştir.

- **auth/**: Kullanıcı girişi ve kimlik doğrulama işlemleri
- **users/**: Kullanıcı yönetimi ve rollerin tanımlanması
- **books/**: Kitap listeleme, stok ve satış bilgileri
- **cart/**: Sepet işlemleri, ekleme, çıkarma ve satın alma
- **demo/**: Demo veri yükleme ve resetleme mekanizması

Backend TypeORM ile veri modelleme yapar. Uygulama başlatıldığında tabloları otomatik oluşturacak şekilde yapılandırılmıştır.

3.3. Frontend Detayları

Frontend **frontend/src/** altında kullanıcı arayüzü bileşenlerine ayrılmıştır.

- **pages/Auth.tsx**: Kullanıcı giriş sayfası
- **pages/AdminDashboard.tsx**: Yönetici paneli
- **pages/CustomerShop.tsx**: Müşteri mağaza sayfası ve sepet görüntüleme
- **components/**: Tekrar kullanılabilen bileşenler
- **hooks/**: Kullanıcı rolüne göre admin/müşteri kontrolleri
- **api/api.ts**: Backend API çağrılarını yöneten servis
- **types/index.ts**: Tip güvenliğini sağlayan TypeScript tanımları

3.4. Kullanıcı Roller

Uygulama iki farklı rol içerir:

- **ADMIN**: Yönetici yetkisine sahip kullanıcılar
- **CUSTOMER**: Kitap alışverişi yapan kullanıcılar

Her role özel sayfa akışları ve yetkilendirme kontrolleri vardır.

3.5. Temel Kullanıcı Akışları

- Kullanıcı giriş yapar

- Rolüne göre admin veya müşteri sayfasına yönlendirilir
- Müşteri kitapları görüntüler, sepete ekler ve satın alır
- Sepet içeriği ve toplam tutar frontend tarafından hesaplanır
- Satın alma işlemi backend'e gönderilir, stok bilgisi güncellenir

3.6. Demo Verileri ve Reset

Projede `backend/src/demo/golden-state.json` dosyası demo verilerini içerir. Bu dosya, veritabanını temiz ve başlangıç durumuna döndürmek için kullanılır.

- Demo reset endpointi: `POST http://localhost:3000/demo/reset`

Bu sayede proje her testten önce veya sunumda yeniden başlatılabilir.

4. Kullanılan Teknolojiler

- Backend: NestJS, TypeScript, TypeORM
- Veritabanı: PostgreSQL
- Frontend: React, TypeScript, Vite
- HTTP: axios
- Bildirimler: sweetalert2
- Kod kalitesi: ESLint

5. Hazır Demo Hesapları

Projeyi hızlıca test etmek için aşağıdaki hesapları kullanabilirsiniz:

- Admin: `admin_zuhre / 123`
- Müşteri: `ahmet_musteri / 123`
- Müşteri: `ayse_okur / 123`
- Müşteri: `mehmet_ogrenci / 123`

6. Notlar

- Backend ve frontend ayrı terminal pencerelerinde çalıştırılmalıdır.
- PostgreSQL bağlantısı doğru değilse uygulama açılmaz.
- Demo reset işlemi ile test verileri her zaman temizlenebilir.
- GitHub linkini kendi proje adresinle güncellemeyi unutmayın.
- Kitap verileri yeniden yüklenir
- Sepet içeriği sıfırlanır

Sunum ve Değerlendirme İçin Notlar

- Proje, bozuk verileri temizleme ve çalışan bir uygulama sağlama amacını yerine getirir.

- Hem admin hem müşteri senaryoları desteklenir.
- Demo reset özelliği sayesinde uygulama her seferinde temiz bir başlangıç alır.
- Frontend ile backend arasında tam iletişim `frontend/src/api/api.ts` üzerinden sağlanır.
- Proje hem front-end hem de back-end seviyesinde modüler ve genişletilebilir bir yapıya sahiptir.

Sorun Giderme

- `backend` başlamıyorsa PostgreSQL bağlantı ayarlarını ve veritabanı adını kontrol edin.
- `frontend` açılmıyorsa `npm run dev` komutunun düzgün çalıştığını ve `5173` portunun kullanılabilir olduğunu doğrulayın.
- `POST /demo/reset` endpointi çalışmıyorsa backend servisinin çalıştığından emin olun.

Ek Notlar

- Backend, TypeORM `synchronize: true` ile çalıştığı için tablolar uygulama açıldığında otomatik oluşturulur.
- Projeyi sunum için hazırlarken backend ve frontend uygulamalarını ayrı terminallerde başlatın.
- Demo reset işlemi, uygulamanın bozulmuş ya da hatalı veri durumlarından hızla kurtulmasını sağlar.